Deribit Price Tracker

Защита тестового задания: junior backend developer

Искандэр Исмагулов

Задача и результат



Задача: раз в минуту получать с Deribit index price btc_usd и eth_usd, сохранять в БД и отдавать через API.

- Клиент Deribit на aiohttp ✅
- Celery: сбор раз в минуту ✅
- PostgreSQL, цена + тикер + UNIX timestamp ✅
- FastAPI: 3 GET-метода с обязательным ticker ✅
- Docker Compose, unit-тесты ✅
- README с Design decisions ✅

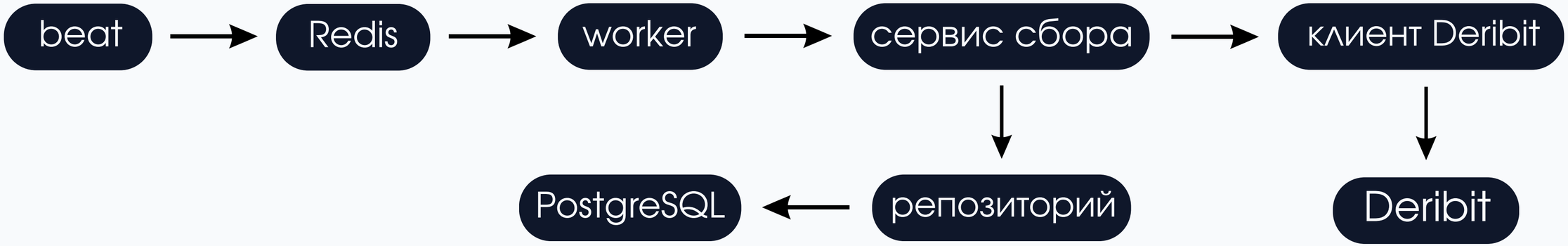
Задача и результат



Требование	Как выполнено
Цены btc_usd, eth_usd раз в минуту	клиент на aiohttp + Celery beat
Тикер, цена, UNIX timestamp	PostgreSQL
API: все данные, последняя цена, по дате	FastAPI, 3 GET-метода, обязательный ticker
Периодика через Celery	worker + beat, Redis как брокер
GitLab, README, Design decisions	есть
Необязательное: тесты, Docker, aiohttp	сделано всё

Архитектура

Поток записи:



Чтение:



зависимости направлены только вниз, каждый слой тестируется отдельно, SQL только в репозитории.

Сбор данных 

- **Расписание:** Beat раз в 60 с ставит задачу, worker выполняет
- **Параллельно:** Обе цены запрашиваются параллельно (asyncio.gather), таймаут 10 с
- **Одна ошибка:** Ошибки сети и разбора превращаются в один DeribitClientError
- **Всё или ничего:** Обе цены сохраняются одной транзакцией: всё или ничего
- **Без retry:** Повторных попыток нет (осознанно)

Хранение данных 

- Индекс (ticker, timestamp) закрывает все три запроса API
- Схема создаётся миграциями Alembic

Поле	Тип	Почему
ticker	varchar	btc_usd / eth_usd
price	NUMERIC(18, 8)	точность, не float
timestamp	BIGINT	UNIX, как в T3

float даёт погрешность, для цен это плохо. Timestamp ставит сервис в момент получения ответов, один на обе валюты цикла.

API 

GET /prices?ticker= → все данные по валюте

GET /prices/latest?ticker= → последняя цена

GET /prices/by-date?
ticker=&date_from=&date_to= → цены за период

- Ответ: {"ticker": "btc_usd", "price": 81075.4, "timestamp": 1792049243}
- Коды: 404 (нет данных у /latest), 422 (нет или неизвестный ticker, период наоборот)
- Ticker это перечисление: валидация и список в Swagger автоматически, один список для API и Celery

Чистый код: ООП и DI 

```python
class PriceCollectionService:
    def __init__(
        self,
        client: DeribitClient,
        repository: PriceRepository,
        tickers: Sequence[str],
        clock: Callable[[], float] = time.time,
    ) -> None:
        self._client = client
        self._repository = repository
        self._tickers = tuple(tickers)
        self._clock = clock
```

- Классы с одной ответственностью: DeribitClient, PriceCollectionService, PriceQueryService, PriceRepository, Database
- Зависимости только через конструктор
- Settings и Database создаются в точках входа: create_app() и worker.py
- На уровне модуля: celery_app, router и константы

В тестах используются фейки и фиксированное время без применения monkeypatch. Экземпляры celery_app и router намеренно оставлены на уровне модуля, так как фреймворки Celery и FastAPI ищут их именно по имени.

Docker и тесты



Запуск:

- docker compose up -d --build
- api, worker, beat, db, redis, migrate
- migrate отрабатывает один раз, остальные его ждут
- Healthcheck у базы и Redis, не root, .env не в образе

Тесты:

- 36 unit-тестов, без сети и без PostgreSQL
- Клиент: подменённые HTTP-ответы
- Сервис сбора: фейки и фиксированное время
- Репозиторий и API: временная SQLite

Ограничение: SQLite отличается от PostgreSQL, поэтому итоговую проверку я делал руками через Docker Compose, в том числе с чистого клона репозитория

Спасибо за внимание!